プログラム説明資料

ゲームの仕組みから操作の手触りまで、一貫して設計する

A型店員のアルバイト

商品を掴み、向きや配置を調整してカゴに収める3Dパズルゲーム。商品の状態や落下を得点に反映し、プレイ中の情報表示とクリア後の評価につなげている。

制作概要	内容
氏名	髙野 佑太
制作体制・期間	プログラマ5名によるチーム制作 / 約2か月
制作背景	1年次の個人制作を基に、3年次のインターンシップでブラッシュアップ。
技術構成	Unity / C# / uGUI / TextMeshPro / Input System / DOTween
主な担当	UI・UXのデザイン / 設計 / 実装、ポーズ・シーン遷移などのコアシステム、レベルデザイン、リファクタリング、企画。

本資料で伝える3つの技術的な強み

① システム連携：商品状態・カゴ判定・得点・結果画面を接続する。
② UI・UX実装：操作状態と判断材料をカーソル周辺に集約する。
③ 共通機能の設計：ポーズ・フェード・サウンドを各画面から再利用する。

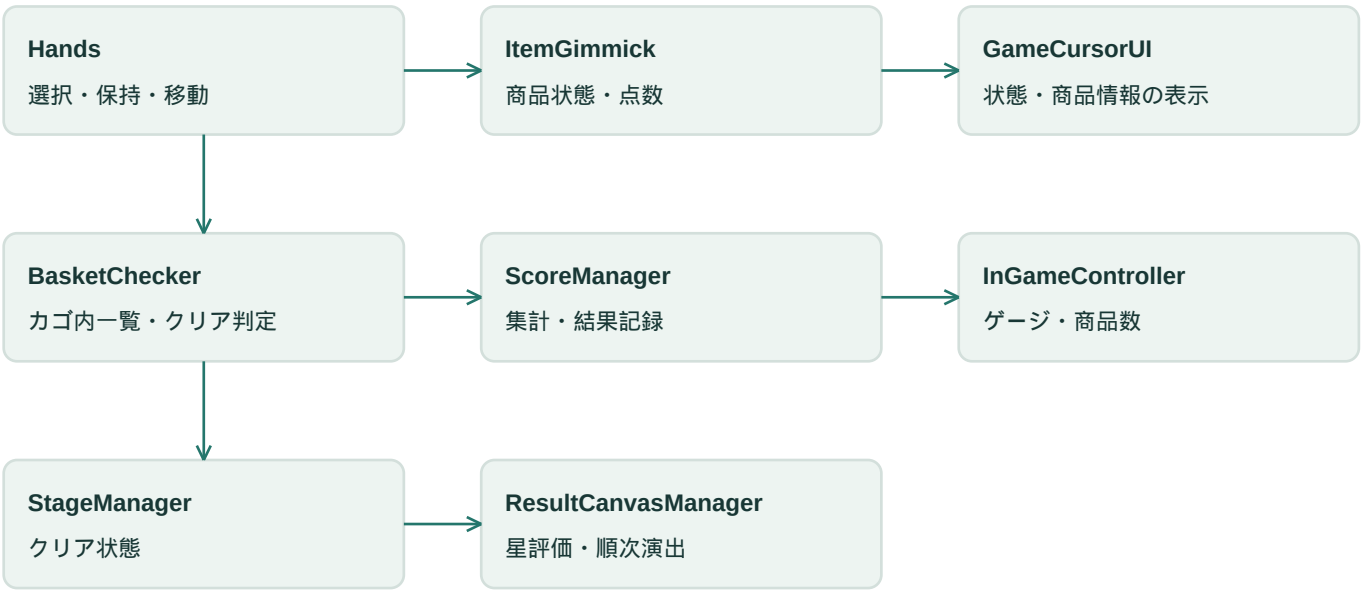
担当範囲の読み方

ScoreManager、PauseManager、FadeManager、AudioManager、各UIクラスを中心に解説する。Hands、ItemGimmick、BasketChecker、FloorCheckerは共同実装・追記対象として扱い、既存ロジックも連携の前提として紹介する。

本資料の読み方：2 ~ 4ページでシステム設計、5 ~ 7ページで得意分野のUI・UX実装、8 ~ 9ページで共同開発と共通機能を紹介する。コード抜粋は提出版ソースに対応。採用理由は処理方式の利点と制約を示す。

全体設計と責務の分担

入力・ゲーム状態・集計・表示をつなぐ



矢印は主なデータ・処理の流れを示す（直接の呼び出しだけを表す図ではない）。

状態を保持する場所と、表示する場所を分ける

ItemGimmickは保持時間・劣化状態・商品点数を管理し、ScoreManagerは集計を担当する。InGameControllerとResultCanvasManagerは、渡された値をゲージ・商品数・星評価へ変換する。この分担により、見た目の調整とゲームルールの変更箇所を把握しやすくしている。

共通機能	呼び出し元と役割
PauseManager	ゲーム内UIや入力から停止・再開を要求する。
FadeManager	タイトル・ポーズ・リザルトなどの画面から遷移を要求する。
AudioManager	シーン読込に応じたBGMと、操作・クリア時のSEを扱う。

採用理由：共通窓口とシーン単位の状態を使い分ける

Instance経由の呼び出しは、複数の画面から共通機能へアクセスしやすい。一方、クラス間の依存は残る。ScoreManagerなどのシーン単位の状態と、DontDestroyOnLoadで維持する共通機能を分け、寿命の違いを意識する必要がある。

> 設計上の制約：完全な疎結合構成ではなく、各クラスが具体的なManagerを参照する。規模が増える場合は、通知イベントや参照注入を使った依存関係の整理を検討する。

実装参照：Hands / ItemGimmick / BasketChecker / ScoreManager / StageManager / 各UIクラス

商品状態を得点と進捗へ変換

仮表示と確定処理を分離し、プレイ中の状況を伝える

処理の流れ

開始時にシーン内の商品点数を合計し、満点を設定する。プレイ中はカゴ内の保持されていない商品を集計して仮表示し、全商品がクリア条件を満たした時点でAddItemResultを呼び、確定スコアとログを記録する。

ScoreManager.cs | L120-141 | 抜粋 (コメント・空行省略 / 長行折返し)

```
120  int tempScore = currentScore;
121  currentItemCounts.Clear();
123  foreach (var item in basketItems)
124  {
125      if (item.GetHeld())
126      {
127          continue;
128      }
131      string name = item.GetItemName();
132      if (currentItemCounts.ContainsKey(name))
133      {
134          currentItemCounts[name]++;
135      }
136      else
137      {
138          currentItemCounts[name] = 1;
139      }
141      tempScore += item.GetItemScore();
```

設計・アルゴリズム	採用理由
仮スコアの再計算	カゴから出た商品や状態変化を次の集計に反映できる。出入りごとの加減算よりも、現在の一覧を正として扱いやすい。
Dictionaryによる種類別集計	商品名をキーに必要数・現在数を対応づけ、商品リスト表示へ渡せる。配列の添字対応を別途管理しなくてよい。
満点からの割合計算	商品数・点数が異なるステージでも、ゲージと星評価を共通の基準で表せる。

具体例：点数が下がるのは「その商品」

1000点の商品を1回落とすと500点になり、同じ商品を再び落とすと250点になる。劣化も商品点数を半減するが、SpoilItemは状態フラグで重複適用を防ぐ。落下処理は確定済みのcurrentScoreから直接減算する方式ではない。

Unity：FindObjectsByTypeで初期の商品一覧を取得し、Image.fillAmountへMathf.Clamp01で0 ~ 1に制限した割合を渡す。C#：ListとDictionaryで結果・商品数を管理する。

計算量と制約：カゴ内商品数nに対する集計は概ねO(n)。現在は毎フレーム再集計し、UI文字列も作り直すため、大量の商品を扱うなら変更時更新を検討する。商品名が同じなら同じ種類として集計される。

ゲームを止め、UIは動かす

時間・入力・画面遷移を分けて制御する

停止は1つの操作では完結しない

PauseManagerはIsPausedを切り替え、Time.timeScaleを0または1に設定し、ポーズCanvasの表示とHandsの有効状態を変更する。再試行・ステージ選択では、PauseMenuControllerが停止を解除してからFadeManagerを呼ぶ。

PauseManager.cs | L76-87 | 抜粋 (コメント・空行省略 / 長行折返し)

```
76  IsPaused = pause;
77  Time.timeScale = IsPaused ? 0f : 1f;
79  if (pauseCanvasInstance != null)
80  {
81      pauseCanvasInstance.SetActive(IsPaused);
82  }
84  Hands hands = FindObjectOfType<Hands>();
85  if (hands != null)
86  {
87      hands.enabled = !IsPaused;
```

処理対象	現在の制御と理由
ゲーム時間	Time.timeScaleでゲーム時間を停止。0にしても全てのコード・入力が止まるわけではないため、操作側の制御も必要。[1]
UIアニメーション	DOTweenのSetUpdate(true)で時間スケールに依存しない更新を指定。ポーズ中もボタンの反応を見せる。[3]
画面フェード	unscaledDeltaTimeで経過時間を進め、WaitForSecondsRealtimeで待つ。停止中でも暗転処理を進められる。[2]

フェードのアルゴリズム：経過時間で透明度を補間

補間率 t = Clamp01(経過時間 / 指定時間)、透明度 = Lerp(開始値, 終了値, t)。フレームごとに固定値を足す方式に比べ、描画回数ではなく指定秒数を基準に変化を定義できる。実際の完了はフレーム単位となる。

暗転 → 実時間で待機 → SceneManager.LoadScene → 明転の順に実行する。CanvasGroup.blocksRaycastsでフェード中のUIクリックを遮るが、独自のゲーム入力まで遮断する設定ではない。

設計を通じた学び：時間停止と入力停止は別に管理する必要がある。現行の入力イベント解除には改善余地があり、enabled=falseだけで停止を保証しない。今後は入力の有効期間と遷移の多重起動防止を共通化する。

実装参照：PauseManager.SetPause / PauseMenuController / FadeManager.TransSceneCoroutine・FadeCoroutine

判断材料をカーソルの近くに置く

状態・残り時間・商品属性をひとつの操作導線にまとめる

ゲームの状態をUIへ渡す

HandsがRaycastで見ている商品を調べ、GameCursorUIへ名前・属性・保持可能時間を渡す。保持中は商品情報を隠して残り時間を表示し、残り3秒以下で赤、それ以外で緑にする。画像は「保持中 または クリック中」で切り替える。

配置アルゴリズム：画面を4領域に分ける

画面中央を基準に左右・上下を判定し、パネルのpivotとオフセットの符号を切り替える。カーソルが右側なら左向き、上側なら下向きにパネルを展開する。

カーソル位置	pivot	パネルの展開方向
左下	(0, 0)	右上
右下	(1, 0)	左上
左上	(0, 1)	右下
右上	(1, 1)	左下

GameCursorUI.cs | L106-107 | 抜粋 (コメント・空行省略 / 長行折返し)

```
106  infoRect.pivot = new Vector2(pivotX, pivotY);
107  infoRect.position = new Vector2(mousePos.x + offsetX,mousePos.y + offsetY);
```

採用理由と、別方式との違い

左右・上下の比較だけで配置方向を決められるため、処理が単純で、オフセットの調整も容易。毎回パネルの矩形を測って画面内へ押し戻す方式と比べて実装量を抑えられる。一方、パネル寸法を使っていないため、画面外へのはみ出しを完全には防げない。

使用する機能	この実装での役割
Input System：ReadValue	ポインター座標を取得する。
Unity：RectTransform.pivot	矩形内の基準点を切り替え、展開方向を変える。[5]
Unity：Screen.width / height	現在の画面中央を判定する。

適用条件：画面座標をRectTransform.positionへ直接代入している。Canvasの描画方式・スケール設定による座標の整合を確認する必要がある。改善時は座標変換とパネル寸法を含む境界補正を行う。

実装参照：Hands.Update / GameCursorUI.Update・SetHoldingCursor・UpdateCursorVisuals

ボタンに押し心地を与える

入力イベントと拡縮アニメーションを対応づける

状態ごとのフィードバック

OnPointerEnterで拡大とSE、OnPointerExitで元の大きさへ復帰、OnPointerDownで押し込みと跳ね返りを実行する。これらはEventSystemのポインターイベントを受けるメソッドであり、毎フレーム自前でボタン範囲を判定する構造ではない。

SelectCursor.cs | L45-57 | 抜粋 (コメント・空行省略 / 長行折返し)

```
45  public void OnPointerDown(PointerEventData eventData)
46  {
48      currentTween?.Kill();
51      Sequence sequence = DOTween.Sequence();
52      sequence.SetUpdate(true);
55      sequence.Append(this.transform.DOScale(clickScale, duration *
        0.5f).SetEase(Ease.OutQuad));
56      sequence.Append(this.transform.DOScale(expansionScale, duration *
        1.5f).SetEase(Ease.OutBack));
57      currentTween = sequence;
```

動作	コード上の初期値	表現すること
ホバー	1.1倍 / 0.15秒	選択対象を視覚的に知らせる。
押し込み	0.9倍 / 0.075秒	押した瞬間の反応を短く返す。
跳ね返り	1.1倍 / 0.225秒 / OutBack	押し込み後に弾む印象を付ける。

採用理由：最新の入力を優先する

各入力でcurrentTweenをKillしてから次の演出を作る。同じTransformへ複数の拡縮を走らせ続けず、退出や再クリックに追従させる。Sequenceで押し込み→跳ね返りを並べることで、演出の順番と秒数をコード上でまとめて調整できる。[3]

Unity：IPointerEnterHandler / IPointerExitHandler / IPointerDownHandlerがイベントの受け口。DOTween：DOScaleが拡縮、SetEaseが変化のカーブ、Appendが順序の指定を担当する。

制約：元のスケールを保存せず1.0へ戻すため、等倍を基準とするボタンを想定している。無効化・破棄時のTween終了処理と、キーボード / ゲームパッド選択への対応は拡張候補。初期値はInspectorで変更可能。

実装参照：SelectCursor.cs / AudioManager.PlaySE

評価を段階的に見せるリザルト

得点割合による評価と、演出の順序制御

評価アルゴリズム

得点割合 r = Clamp01(totalScore / maxScore) を求め、高い閾値から順に比較する。初期設定では100%で星3、66%以上で星2、33%以上で星1、それ未満は星0。割合で評価するため、ステージごとの商品数の違いを吸収できる。

ResultCanvasManager.cs | L156-172 | 抜粋 (コメント・空行省略 / 長行折返し)

```
156  float scoreRatio = Mathf.Clamp01((float)totalScore / maxScore);
157  int starCount = 0;
159  if (scoreRatio >= threeStarThreshold)
160  {
161      starCount = 3;
162  }
163  else if (scoreRatio >= twoStarThreshold)
164  {
165      starCount = 2;
166  }
167  else if (scoreRatio >= oneStarThreshold)
168  {
169      starCount = 1;
170  }
172  return starCount;
```

表示順序と時間 (星3つ・各参照が有効な場合)

順番	演出	所要時間
1	背景をOutBackで拡大	0.5秒
2	星を1つずつ拡大	0.3秒 × 3
3	間を置く → スコアをフェード表示	0.3秒 + 0.5秒
4	間を置く → ステージ選択ボタンを拡大	0.3秒 + 0.3秒

採用理由：評価と次の行動を順番に提示する

背景の完了コールバックでSequenceを開始し、AppendCallbackで表示状態を変えてからAppendで演出を再生する。評価を見る時間を作り、最後に次の行動を提示する。時間を足し合わせると上記条件で約2.8秒となる (実測値ではない) 。

DOTweenのSequence単位でSetUpdate(true)を指定し、子Tweenを含めて時間スケールから独立させる。[3] 星評価の閾値は[SerializeField]で調整できる。

現状の範囲：ShowResultUIはdropCountとlogsを受け取るが、現行コードではその内容を画面に表示していない。結果ログの生成と、結果ログの表示は区別する。

商品操作と判定をシステムへ接続

共同実装・追記対象：既存ロジックと自分の連携処理を区別する

既存の操作ロジック：画面から一定高さの位置を求める

HandsではScreenPointToRayで視線方向のRayを作り、水平面 y = HOLD_HEIGHT との交点へ商品を移動する。Ray を O + tD とすると、t = (HOLD_HEIGHT　O.y) / D.y。マウスを、カメラから一定距離の点ではなく一定高さの操作面へ対応づけられる。[6]

Hands.cs | L183-188 | 抜粋 (コメント・空行省略 / 長行折返し)

```
183  if (Mathf.Abs(ray.direction.y) > 0.0001f)
184  {
185      float t = (fixedY - ray.origin.y) / ray.direction.y;
186      Vector3 worldPos = ray.origin + ray.direction * t;
187      worldPos.y = fixedY;
188      heldTransform.position = worldPos;
```

追記・連携として説明するポイント

対象	連携内容
Hands → GameCursorUI	選択中商品の情報を表示し、保持中の残り時間とカーソル画像を更新する。
Hands → Physics	保持商品とカゴ内商品のColliderの組み合わせにIgnoreCollisionを設定し、操作中の干渉を抑える。[7]
ItemGimmick → PopupManager	状態悪化時に「Bad...」を表示。popupShownで繰り返し生成を防ぐ。
BasketChecker / FloorChecker → ScoreManager	クリア時に商品結果を確定する。落下時は商品点数を半減し、ポップアップと机上への復帰へつなぐ。

クリア条件を「入った瞬間」だけで決めない

ItemGimmickはカゴ内で線速度・角速度の大きさが共に0.1以下の状態を計測し、初期設定で2秒続くとisOKを立てる。BasketCheckerはisOKかつ保持されていない商品数が全商品数と一致することを確認する。一時的な通過や移動中の確定を避ける設計として説明できる。

共同開発での貢献：既存の操作・判定を読み解き、カーソル表示、ポップアップ、スコア集計へ接続した。交点計算や静止判定は連携先の仕組みとして紹介しており、単独実装の主張には含めない。なお静止成立後のisOKは、再び動くだけでは解除されない。

共通機能で体験の連続性を保つ

シーンの切り替えと、BGM・設定の扱いをそろえる

BGMの切り替え方

AudioManagerはシーン名とAudioClipの対応配列を持ち、sceneLoadedイベントで該当する曲を選ぶ。FadeManagerからは遷移前に次の曲を問い合わせ、異なる曲ならフェードアウトする。同じ曲が再生中ならPlayBGMを早期終了し、曲頭へ戻す処理を避ける。

AudioManager.cs | L145-157 | 抜粋 (コメント・空行省略 / 長行折返し)

```
145  if (bgmSource.clip == clip && bgmSource.isPlaying)
146  {
147      return;
148  }
151  if(fadeCoroutine != null)
152  {
153      StopCoroutine(fadeCoroutine);
154  }
157  fadeCoroutine = StartCoroutine(FadeBGM(clip, fadeDuration));
```

実装	採用理由・役割
シーン名と曲の対応配列	Inspectorから対応を設定でき、各シーンに選曲ロジックを分散させずに済む。探索は設定数mに対してO(m)。
BGMとSEのAudioSourceを分離	BGMの曲・フェードと、操作SEの再生を別々に扱える。SEはPlayOneShotを使う。
PlayerPrefsで音量を保持	BGM_Volume / SE_Volumeのキーで値を読み書きし、次の起動時の設定に利用する。[9]
sceneLoadedの購読と解除	OnEnableで登録し、OnDisableで同じメソッドを解除する。イベントの有効期間を明示する。

補助UIも同じ基盤を利用する

TutorialManagerは表示済みキーを読み、未表示ならフェード時間分を実時間で待ってCanvasを生成する。UI初期化のため1フレーム待った後にゲーム時間とHandsを停止する。表示済み管理により、説明を毎回繰り返すかどうかを制御する。

設計を通じた学び：共通Managerへまとめるだけでなく、音量を適用する箇所や、対応するBGMがない場合の挙動まで統一する必要がある。今後はSE音量の二重適用を解消し、設定値と聴感の関係を調整しやすくする。

実装参照：AudioManager / FadeManager / SettingsMenuController / TutorialManager・TutorialController

制作を通じて発揮した強み

システム全体を捉え、UI・UXの品質へつなげる

機能をつなぎ、プレイヤーへ伝わる形にする

本作では、UIの見た目やアニメーションに加えて、ポーズ・画面遷移・音声・スコアなどの基盤機能を担当した。商品操作や判定を扱う既存コードにも追記し、ゲームの状態をプレイヤーが理解できる表示へつなげた。

強み	本作で示した実装
全体を見たシステム構築	商品状態から仮スコア・確定スコア・リザルトまで、複数クラスにまたがるデータの流れを接続した。
UI・UXを具体化する力	カーソルへの残り時間表示、商品情報の配置、ボタンの押し込み、リザルトの段階表示を実装した。
共同開発での連携	他メンバーの操作・判定処理を前提に、表示と集計を追加。共同実装と担当機能を区別して説明できる。
調整を支える実装	演出時間・拡縮率・評価閾値などを設定項目にし、コードのロジックとパラメータ調整を分けた。

設計の振り返りと、次に改善すること

① 状態の終了処理をそろえる
入力の購読・解除を同じハンドラーで対にし、クリック解除・時間切れ・ポーズなど、どの経路でも保持状態と衝突設定を復元する。[10]

② 表示更新を必要なタイミングへ絞る
毎フレームの商品集計・文字列生成から、商品数や状態が変わった時の更新へ整理する。変更前後を計測し、必要な最適化を判断する。

③ 共通機能の寿命を統一する
常駐Managerの重複防止、ポーズとチュートリアルの停止管理、遷移中の入力制御を一貫した方針にまとめる。

品質を確認する観点

連打や素早いホバー移動、保持中のポーズ、シーン往復、画面四隅での情報表示、星評価の閾値前後など、通常操作に加えて状態の切り替わりを重点的に確認する。

確認範囲：本資料の作成では提出ソースを静的に確認した。ここに示す品質確認は検証観点であり、今回の実機テスト結果や性能計測結果を示すものではない。

Unity固有の機能と使い分け

本文の処理を理解するための補足

機能・分類	本作での使い方	使う際の要点
Awake / Start Unityメッセージ	Instanceや入力の初期化 / 商品集計やUI初期化。	別オブジェクトのAwakeの実行順へ暗黙に依存しない。[4]
Update Unityメッセージ	カーソル追従、保持時間、カゴ内集計。	毎フレーム必要な処理と、変更時だけ必要な処理を区別する。
OnEnable / OnDisable Unityメッセージ	入力の有効化やシーンイベントの購読・解除。	同じハンドラーを登録・解除する。[4][10]
OnTriggerEnter / Exit Unityメッセージ	カゴ・床への出入りを商品状態へ反映。	Collider、Rigidbody、レイヤーの構成が前提となる。
DontDestroyOnLoad Unity API	音声・フェード・ポーズUIなどを維持。	永続化と重複防止は別処理として設計する。
StartCoroutine / yield Unity API・C#構文	フェードやチュートリアルの待機。	別スレッド化ではなく、処理を中断・再開する仕組み。
[SerializeField] Unity属性	privateな閾値・時間・UI参照を設定対象にする。	コード上の初期値とInspectorの保存値は一致するとは限らない。
ScreenPointToRay Unity API	マウスの画面座標からRayを作る。	Raycastによる物体選択と、水平面との交点計算に使う。[6]
ReadValue / started Input System	座標の取得 / 掴む・ポーズ入力の開始通知。	Unity標準のMonoBehaviourメッセージとは別の入力API。[8]
Sequence / DOScale DOTween	順次演出 / 拡縮アニメーション。	外部ライブラリ。自作の補間アルゴリズムとして説明しない。[3]

自作メソッドは「ゲームの意味」を表す

SetHoldingCursor、AddItemResult、SetGameClearなどは自作メソッド。Unityやライブラリの API を組み合わせ、「商品を持つ」「得点を確定する」「ゲームをクリアする」といったゲーム固有の処理へまとめている。

説明の粒度：APIの名前を列挙するだけでなく、呼び出す条件・入力値・変更される状態・終了時の後始末まで示す。これにより、機能を使えることと設計を理解していることを同時に伝える。

参照資料と確認範囲

ソースとの対応を追える説明資料にする

一次資料

添付「髙野佑太_作品①.zip」内のソースコード、プログラム説明書（4ページ）、作品説明資料（5ページ）を使用。本文のコード抜粋は添付ソースの行番号を併記し、コメント・空行・表示上の折り返しのみ調整した。

担当範囲は既存説明資料とファイルヘッダーを根拠として整理した。Gitの差分履歴は含まれていないため、共同実装クラスの行単位の担当者は特定していない。

公式リファレンス（2026年9月29日参照）

[1] Unity：Time.timeScale

[2] Unity：Time.unscaledDeltaTime

[3] DOTween：Documentation

[4] Unity：MonoBehaviour

[5] Unity：RectTransform.pivot

[6] Unity：Camera.ScreenPointToRay

[7] Unity：Physics.IgnoreCollision

[8] Unity Input System：InputAction

[9] Unity：PlayerPrefs

[10] Microsoft：C#イベントの登録と解除

各資料名はリンク。Unity APIの説明はUnity 6.0、Input Systemは1.11のリファレンスを参照したが、本作のインストール済みバージョンを示すものではない。

記載の判断基準

区分	本資料での扱い
実装済み	添付コードにある分岐・計算・API呼び出しを説明。
採用理由	コードの構造から説明できる利点・代替方式との違いを整理。未確認の開発エピソードは記載しない。
結果・効果	コード上の挙動と、設計として期待する効果を区別。快適性の比較評価や性能改善率は記載しない。
改善案・検証計画	現行コードへ反映済みとは扱わず、今後の対応として記載。